SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Grama Jyothi Billing Program

Field	Entry
Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Design, Implement and Validate a Reliable Billing Program
GitHub Repository URL	https://github.com/bavadarini28 /CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

TASK 4 – Use Macros Safely

Aim

To write small macros correctly and understand their pitfalls.

```
#define RUPEES_TO_PAISE(r) ((long)(r) * 100L)
#define MAX(a,b)   (((a) > (b)) ? (a) : (b))
#define SQUARE(x)  ((x) * (x))
#define SQ_BAD(x)  x*x
```

Test and output:

```
int a=3;
printf("%d %d", SQ_BAD(a+1), SQUARE(a+1));   /* prints: 7 16 */
int i=5,j=3; int m = MAX(i++, j);          /* m=6 i=7 */
```

Programiz
C Online Compiler

main.c Clear Run ▶ Output

```c
#include <stdio.h>

#define RUPEES_TO_PAISE(r) ((long)(r) * 100L)

#define MAX(a,b) (((a) > (b)) ? (a) : (b))

#define SQUARE(x) ((x) * (x))

#define SQ_BAD(x) x*x

int main()
{
    printf("Rupees to Paise: %ld\n", RUPEES_TO_PAISE(50));

    printf("Maximum number: %d\n", MAX(10, 20));

    printf("Square: %d\n", SQUARE(5));

    printf("Bad Square: %d\n", SQ_BAD(5 + 2));

    return 0;
```

```
Rupees to Paise: 5000
Maximum number: 20
Square: 25
Bad Square: 17
```

- Expansion: gcc -E replaces macros by text. SQ_BAD(a+1) becomes a+1*a+1 = 3+3+1 = 7 because * binds tighter than +. SQUARE adds parentheses: ((a+1)*(a+1)) = 16.
- Double evaluation: MAX(i++, j) expands to ((i++) > (j)) ? (i++) : (j). Since 5 > 3, i++ runs twice: m is 6 (not 5) and i ends at 7.
- Use a function instead when arguments have side effects, when type checking is needed, when the body is long, or when debugging/stepping matters.
- Parenthesise every parameter and the whole expansion.

Alternative Concept – static inline functions

```
static inline int max_int(int a,int b){ return a>b ? a : b; }
static inline int square(int x){ return x*x; }
```

Arguments are evaluated once (max_int(i++, j) is safe) and type-checked, but are fixed to one type (int) unless _Generic is used. Time: a call costs O(1) either way; with optimisation the compiler inlines the function, so run time equals the macro; at -O0 a real call adds a few cycles. Space: a macro is copied at each of k uses, so code size is O(k × body); an inline function is also copied where inlined, but if not inlined there is one copy, O(1 + k × call).

Result: Macros are fully parenthesised and both pitfalls are demonstrated.